IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Manuscript prepared on May 25, 2026.

Digital Object Identifier

A Unified Framework for Designing Swarm Robots Using a Model-Based Systems Engineering Approach

HOANG ANH PHAM [1], VALENTIN GIES[2], and THIERRY SORIANO[3],
[1]Faculty of Electronics Engineering 1, Post and Telecommunications Institute of Technology, Hanoi, Vietnam
[2]Laboratory IM2NP, University of Toulon, Toulon, France
[3]Laboratory COSMER, University of Toulon, Toulon, France

Corresponding author: Hoang Anh Pham (e-mail: anhph@ptit.edu.vn).

ABSTRACT Multi-robot applications are difficult to validate when system models, simulation assets, and robot software evolve separately. This paper presents a model-based systems engineering (MBSE) workflow that connects SysML artifacts to an executable ROS 2 implementation for a two-TurtleBot4 target-acquisition mission. The workflow defines requirements, behavioral views, software architecture, hardware architecture, and data-flow views, then maps them to ROS 2 nodes, topics, launch files, DDS network settings, and validation metrics. The implemented case study uses exactly two canonical robot identities, `/robot1` and `/robot2`, while the physical TurtleBot4 Lite deployment boundary records the required FastDDS, `ROS_DOMAIN_ID`, Simple Discovery, and `/cmd_vel_unstamped` interfaces. A coordinator assigns the closest robot with fresh odometry as attacker, assigns the second robot a support pose near the target, publishes observable task state and events, and records the result in CSV metrics. A stable Gazebo simulation with TurtleBot4 visual models validates the executable chain before physical deployment. The latest run completed the mission in 8.028 s with no role switches and a minimum inter-robot separation of 0.6985 m. The contribution is not a general swarm benchmark, but a traceable sim-to-real workflow whose scope is deliberately aligned with the available two-robot platform.

INDEX TERMS Model-based systems engineering, SysML, TurtleBot4, ROS 2, swarm robots, sim-to-real validation

I. INTRODUCTION

Model-based systems engineering (MBSE) is increasingly used to manage the interaction between requirements, architecture, behavior, and verification in complex engineered systems [1], [2]. Robotics brings the same integration problem into a smaller but more dynamic setting: the model must remain consistent with sensors, computation, communication, motion control, simulation, and hardware deployment. This consistency is especially important for multi-robot systems because a coordination error may be caused by an ambiguous requirement, an incomplete architecture model, a ROS 2 namespace or `ROS_DOMAIN_ID` mismatch, a simulator limitation, or a hardware safety constraint.

This paper studies that problem through a deliberately bounded case study: two TurtleBot4 robots performing a target-acquisition mission. One robot is assigned as the attacker and moves toward the target. The second robot is assigned as the supporter and moves to an offset pose near the same target. The number of robots is fixed by the available physical platform, and the paper does not claim validation beyond this two-robot setting.

The central research question is how to keep SysML models useful after the transition from conceptual design to executable ROS 2 code. Rather than treating SysML diagrams as documentation only, the proposed workflow uses them as traceability anchors for the implementation: requirements are mapped to nodes, topics, launch files, runtime events, DDS settings, and metrics. The same canonical robot identities are used in simulation and in the physical deployment adapter so that the design boundary is explicit.

The contributions of this paper are:

- a SysML-based workflow that connects requirements, activity, structure, software architecture, hardware architecture, and data-flow views for a two-TurtleBot4 mission;
- a ROS 2 implementation that realizes the modeled

behavior through `/robot1` and `/robot2` canonical identities, observable roles, target goals, task state, and metrics;
- a sim-to-real deployment boundary in which simulation is executable by default and physical motion commands require explicit enablement;
- an evaluation run showing that the modeled requirements can be traced to a completed simulation result.

The rest of the paper is organized as follows. Section II reviews related work on swarm robotics, MBSE, SysML, and ROS-based systems. Section III presents the proposed MBSE-to-ROS 2 workflow. Section IV describes the two-TurtleBot4 case study and validation results. Section V concludes the paper and identifies remaining limitations.

II. RELATED WORK

A. SWARM ROBOTICS AND MULTI-ROBOT COORDINATION

Swarm robotics studies how multiple robots can coordinate through task allocation, navigation, communication, and local decision-making. Surveys of swarm robotics tasks emphasize behaviors such as aggregation, flocking, foraging, deployment, collaborative manipulation, and task allocation [3]. From a swarm engineering perspective, these behaviors must be designed, verified, and maintained through systematic processes rather than through isolated controller implementations [4]. Mission-oriented architectures for unmanned swarms further show that reusable mission structure is useful when the same coordination logic must be adapted to different platforms or scenarios [5].

The present work does not attempt to cover all swarm behaviors. It selects one representative coordination problem–role allocation for target acquisition–and uses it to demonstrate traceability from system requirements to executable ROS 2 behavior. This narrower scope makes the validation boundary explicit and keeps the experiment aligned with the available two-TurtleBot4 platform.

B. MBSE AND SYSML FOR EXECUTABLE ENGINEERING

MBSE replaces scattered document artifacts with linked models that capture requirements, structure, behavior, and verification intent [1], [2]. SysML has been used to structure engineering knowledge at multiple abstraction levels, allowing high-level decisions to remain connected to lower-level design elements [6]. Systems-thinking work also emphasizes the transition from problem capture to formal SysML models, reducing information loss between early design and engineering artifacts [7].

Several domains have demonstrated the value of MBSE for architecture and traceability, including aircraft systems [8], [9], satellite communication systems [10], and digital twins for industrial robots [11]. These works show that MBSE is most useful when the model supports downstream analysis or implementation. The contribution here is to apply the same principle to a small mobile-robot team and to keep the SysML artifacts connected to concrete ROS 2 entities and measured simulation results.

C. ROS-BASED ROBOTIC SYSTEMS

ROS and ROS 2 provide a practical middleware layer for robotic software, but the flexibility of nodes, topics, parameters, and launch files can make architecture-level consistency difficult to maintain. A SysML-based metamodel for ROS systems has been proposed to represent both runtime systems and developer workspaces [12]. Supervisory controller synthesis for ROS-based applications also illustrates the need to connect requirements with executable robotic behavior [13]. This paper follows the same motivation from a traceability perspective: every requirement in the case study is linked to a ROS 2 artifact and a runtime metric.

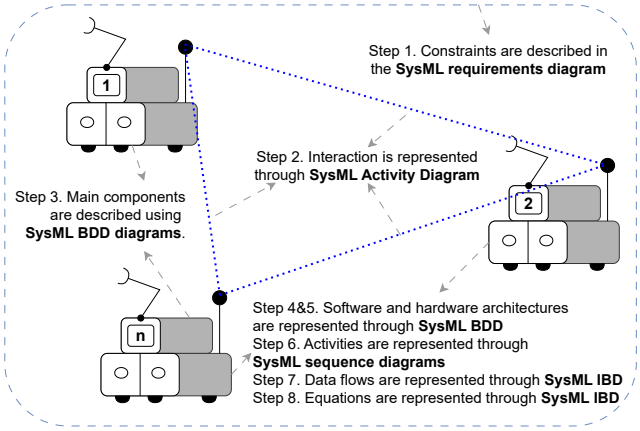

FIGURE 1. Illustrating the use of SysML in the design of swarm robotics

D. SYSML FRAMEWORK FOR SWARM ROBOTS

The objective of using SysML is to provide a standardized method for modeling and integrating the hardware, software, and behavioral aspects of a coordinated robot team (see Figure 1). In this paper, the validated team contains two TurtleBot4 robots. The same modeling pattern can be reused for larger teams, but larger teams are outside the validation boundary of the present work.

- **Modeling language:** SysML is used because it represents requirements, behavior, structure, and inter-component relationships within one consistent notation.
- **Tool independence:** The workflow does not depend on a specific SysML tool; it requires only support for requirement diagrams, block definition diagrams, activity diagrams, sequence diagrams, and internal block diagrams.
- **Modeling conventions:** Requirement identifiers, robot identities, ROS 2 topic names, DDS domain settings, and validation metrics are kept consistent so that each diagram can be traced to an executable artifact.

III. PROPOSED MBSE-TO-ROS 2 WORKFLOW

The proposed workflow links a SysML model to executable ROS 2 artifacts. It is organized around four questions: what must the two-robot system do, how is the mission behavior represented, which software and hardware elements realize the behavior, and how is the result verified. Figure 2 captures the top-level requirements, Figures 3–7 provide behavioral and structural views, and Figures 8–10 connect the model to implementation and platform concerns.

A. REQUIREMENT MODEL AND TRACEABILITY

The requirement model defines the validation boundary before implementation. The mission contains exactly two TurtleBot4 robots. They cooperate on one target-acquisition task, assign attacker and supporter roles from fresh odometry, publish observable state, validate the behavior in simulation before physical deployment, and preserve a documented mapping to the physical TurtleBot4 Lite network configuration. This early boundary is important because it prevents the model from implying a robot team size or capability that is not available in the experimental platform.

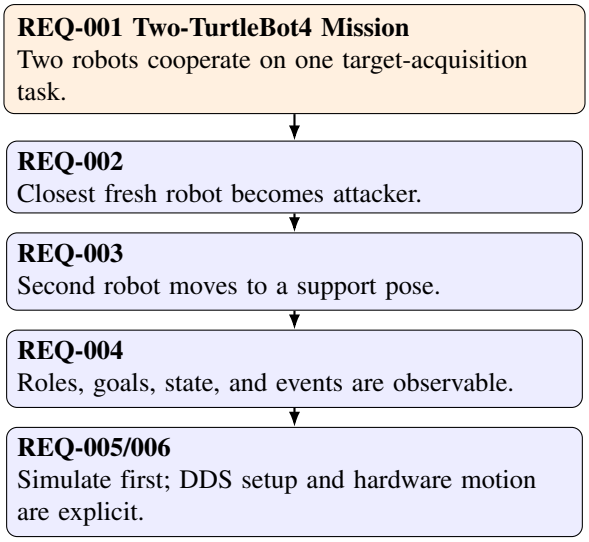

FIGURE 2. High-level requirements for the implemented two-TurtleBot4 case study

Each requirement is later mapped to a ROS 2 artifact: the coordinator node realizes role allocation, goal topics realize target and support assignments, launch files define simulation and physical modes, and CSV metrics provide evidence for verification. This gives the paper a traceable path from requirement to runtime result rather than a diagram-only model.

B. BEHAVIORAL AND STRUCTURAL SYSML VIEWS

The behavioral model uses a general swarm-task taxonomy as context, but selects task allocation, navigation, communication, and metric reporting as the executable subset for this study. Figure 3 shows the broader task model, while Figure 4 shows the activity pattern used by the mission: sense state, exchange information, allocate a role, execute the assigned action, and report completion.

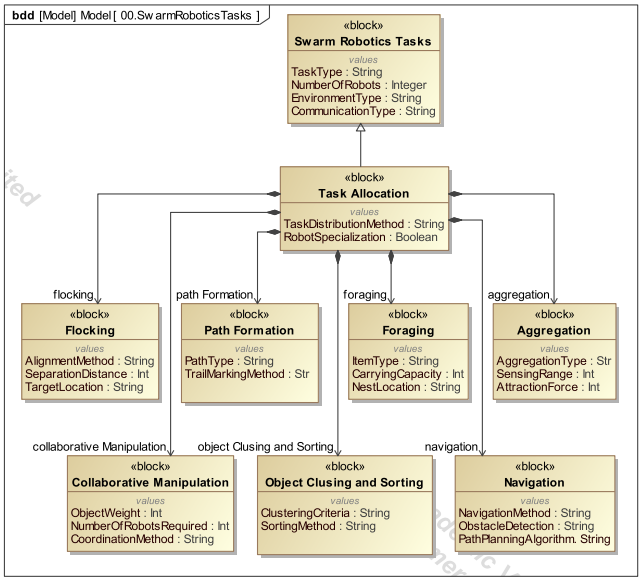

FIGURE 3. Swarm robotics task model used to locate the implemented behavior

The structural view decomposes an individual robot into sensing, communication, control, and actuation responsibilities. In the TurtleBot4 implementation, these responsibilities map to odometry and sensor streams, ROS 2 communication, coordinator decisions, and velocity-command interfaces. Figure 5 gives the block-level view, and Figures 6 and 7 show the interaction sequence used when a robot receives a task, senses its environment, executes a motion command, and reports status.

C. SOFTWARE ARCHITECTURE MAPPING

The software architecture is deliberately small enough to be executable and inspectable. The coordinator consumes target and odometry information, assigns roles, publishes goals, and emits task state. The simulation-only goal follower converts goals into velocity commands. The metrics logger subscribes to task state, events, and odometry to record validation data. In physical deployment, the coordinator and metrics nodes can run without commanding robot motion; direct velocity following is disabled unless explicitly enabled for controlled testing and mapped to the TurtleBot4 Lite `/cmd_vel_unstamped` interface.

The architecture in Figure 8 is mapped directly to the ROS 2 package. The **ControlSystem** is the `paper_swarm_coordinator` node. The **MissionState** is represented by `/target_pose`, `/robot*/odom`, `/swarm/task_state`, and `/swarm/events`. The **MotionControlModule** is represented by the goal follower in simulation and by the external navigation or safety stack when the physical robots are used. For TurtleBot4 Lite hardware, the command endpoint is treated as

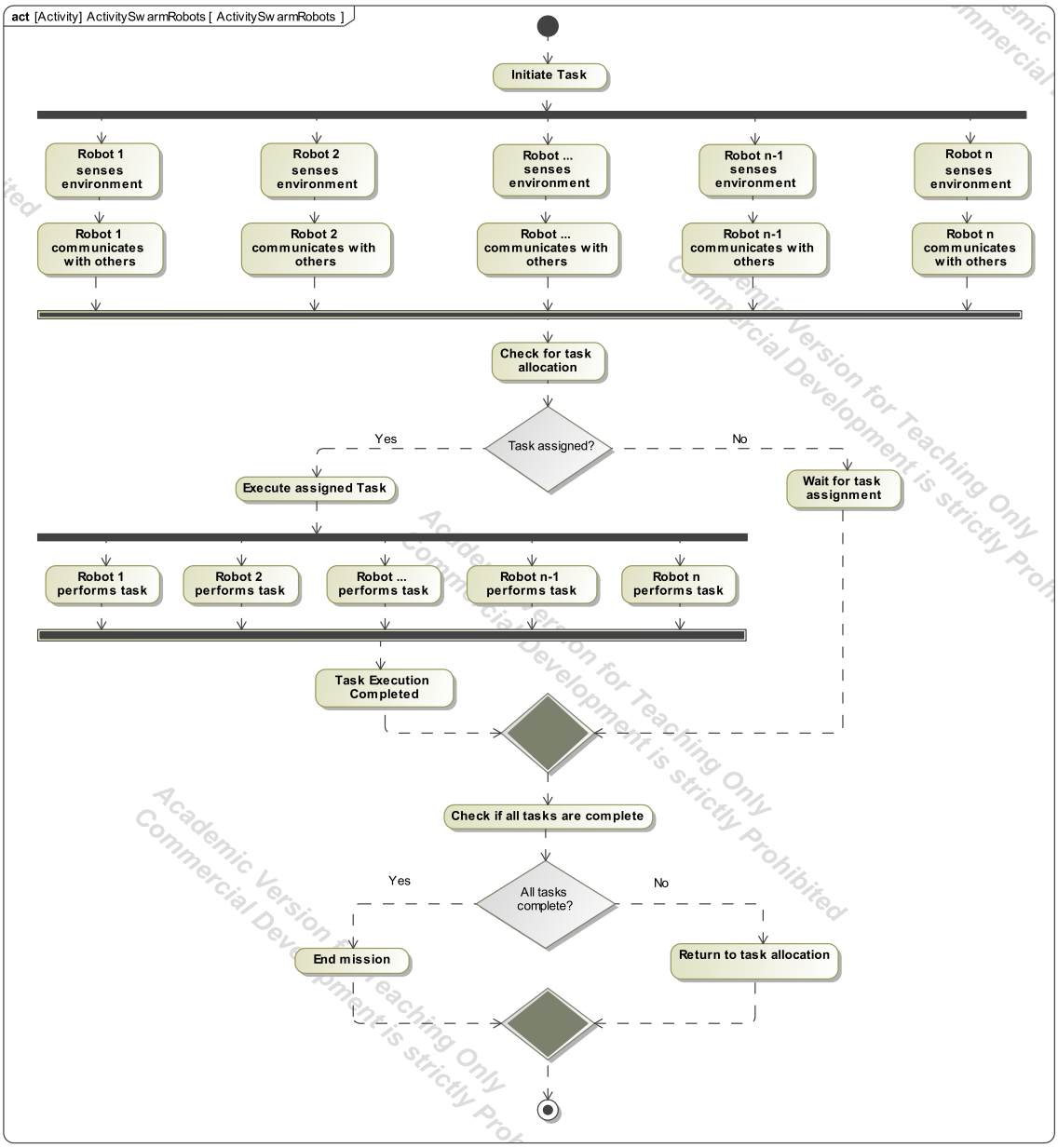

FIGURE 4. Activity model for task allocation and coordinated execution

`/robot*/cmd_vel_unstamped` after namespace or bridge adaptation.

D. HARDWARE AND DATA-FLOW VIEWS

The hardware model identifies the platform assumptions that the software must respect: differential-drive motion, onboard computing, battery power, lidar/vision/IMU sensing, and wireless communication. The implementation uses TurtleBot4-class interfaces instead of a custom robot design, but the hardware view remains useful because it documents which physical resources are expected by the software architecture.

The Internal Block Diagram (IBD) in Figure 10 describes how data moves between sensing, processing, communication, power, and actuation elements. The corresponding software data-flow view in Figure 11 connects those platform-level flows to the ROS 2 topics used by the mission. Together, the diagrams justify why odometry freshness, topic identities, DDS domain configuration, and command gating are treated as explicit requirements.

IV. CASE STUDY: TWO TURTLEBOT4 TARGET ACQUISITION

A. EXPERIMENTAL OBJECTIVE

The case study evaluates whether the SysML requirements can be carried through to an executable ROS 2

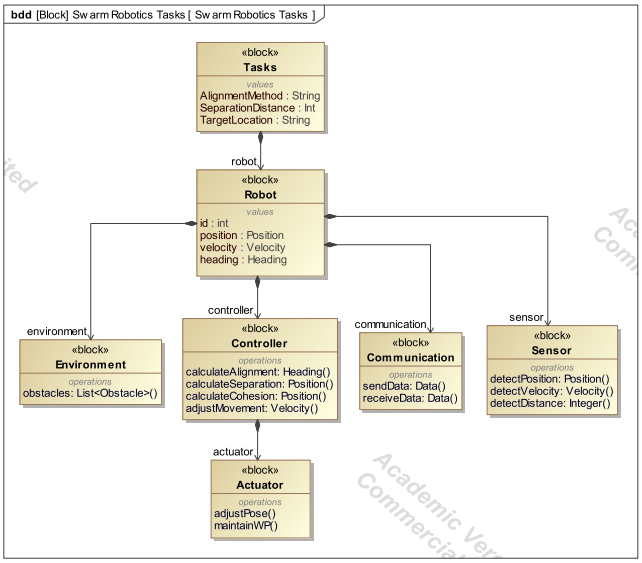

FIGURE 5. Robot-level structural decomposition for sensing, control, communication, and actuation

mission without changing the system boundary. The experimental platform contains two TurtleBot4 robots, represented inside the model and simulator by the canonical identities `/robot1` and `/robot2`. A physical TurtleBot4 Lite, however, is commonly configured as a single robot with FastDDS, `ROS_DOMAIN_ID=0`, Simple Discovery, an empty user namespace, and root topics such as `/odom` and `/cmd_vel_unstamped` [14], [15]. Therefore, a two-robot physical deployment must either assign unique namespaces in the shared ROS graph or bridge/remap two single-robot graphs into the canonical `/robot1` and `/robot2` interfaces. The mission is target acquisition: one robot moves to the target as attacker, while the other maintains a nearby support pose. This task is small, but it exercises the essential MBSE links used throughout the paper: requirement decomposition, role allocation, communication, software-to-hardware mapping, simulation validation, and physical deployment gating.

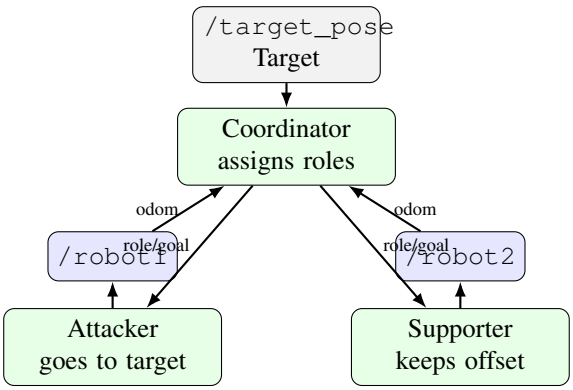

FIGURE 12. Implemented two-TurtleBot4 attacker/supporter target-acquisition scenario

B. ROLE ALLOCATION LOGIC

Let p_T be the target position and p_i be the latest odometry position for robot $i \in \{1,2\}$. A robot is eligible only if its odometry age Δt_i is below the freshness threshold $\tau_{\max}$. The attacker is selected as

$$a = \arg \min_{i\in\{1,2\},\ \Delta t_i \le \tau_{\max}} \|p_i - p_T\|_2 .$$

The other robot becomes the supporter. The attacker goal is the target itself, while the supporter goal is offset from the target:

$$g_a = p_T, \qquad g_s = p_T + \begin{bmatrix} d_x \\ d_y \end{bmatrix} .$$

A hysteresis margin is used to avoid unstable role switching when both robots are at similar distances from the target. This logic directly implements REQ-002 and REQ-003.

C. RUNTIME SEQUENCE

The runtime sequence follows the activity model in Section III and is limited to target acquisition, role allocation, support positioning, and metric logging:

1) A target is published on `/target_pose`.
2) Both TurtleBot4 robots publish fresh odometry on the canonical interfaces `/robot1/odom` and `/robot2/odom`, either directly through namespaces or through a physical remapping layer.
3) The coordinator estimates distance from each robot to the target.
4) The closest fresh robot becomes attacker, with a hysteresis margin to avoid unstable role switching.
5) The second robot becomes supporter and receives a support goal offset from the target.
6) Runtime state and events are published for debugging, traceability, and metric logging.
7) Metrics are written to CSV files for validation and regression checks.

D. TRACEABILITY TO ROS 2 CODE

The SysML requirements are implemented in the `paper_tb4_swarm` ROS 2 package. Table 1 summarizes the traceability link from each requirement to the executable artifact used in the case study.

TABLE 1. Traceability between paper requirements and implementation

Req.	Design intent	Implementation
REQ-001	Two robots cooperate on one target task	`mbse_model.py`, `coordinator.py`
REQ-002	Closest fresh robot becomes attacker	`coordinator.py` role selection
REQ-003	Second robot supports near target	`support_offset_x/y` parameters
REQ-004	Roles, goals, states, events are observable	`/robot*/assigned_role`, `/robot*/goal_pose`, `/swarm/*`
REQ-005	Simulation before hardware	`stable_two_tb4.launch.py`, CSV metrics
REQ-006	Physical DDS setup and motion require explicit enablement	`real_two_tb4.launch.py`, `ROS_DOMAIN_ID`, `/cmd_vel_unstamped`

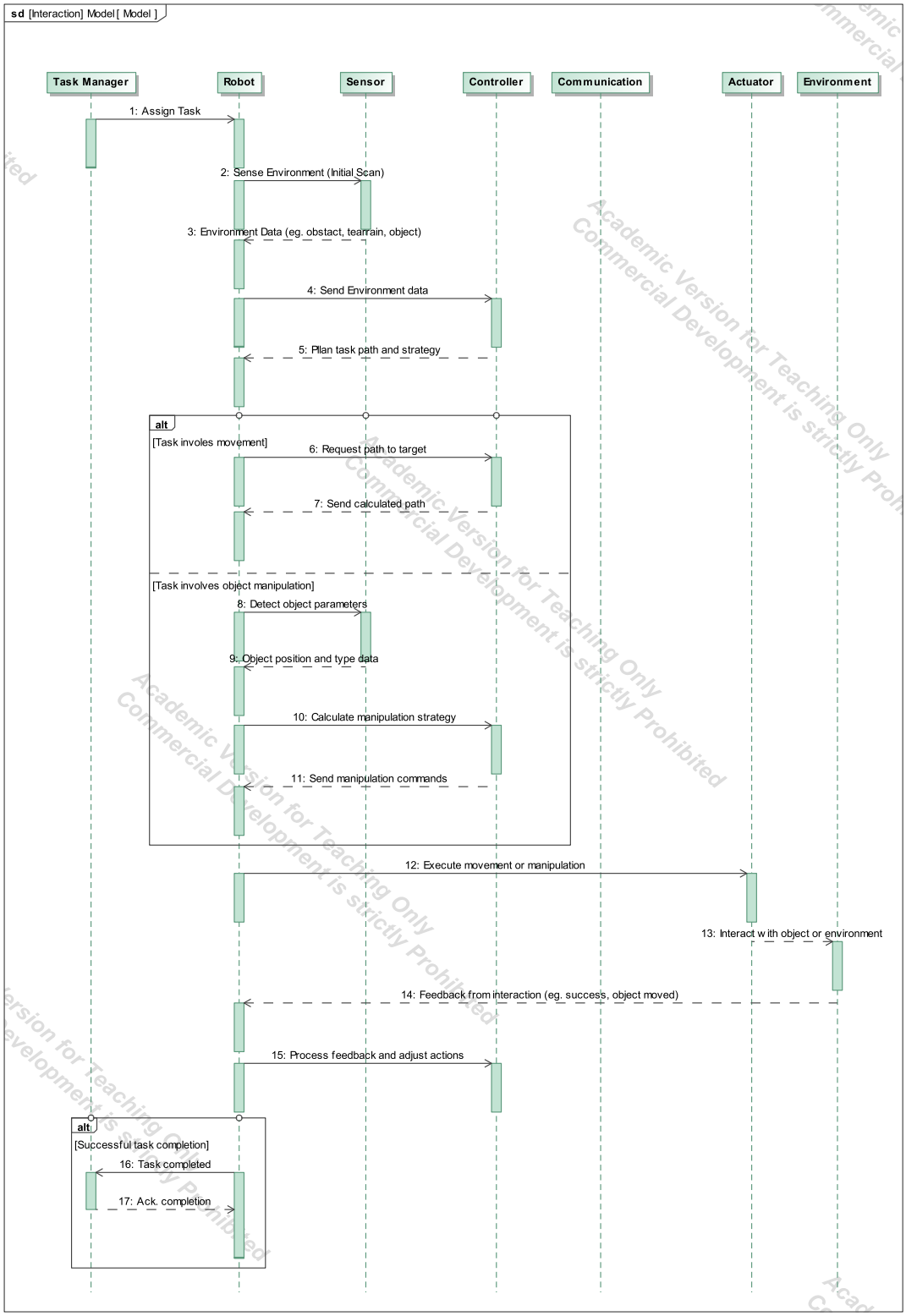

FIGURE 6. Robot interaction sequence for task assignment and sensing

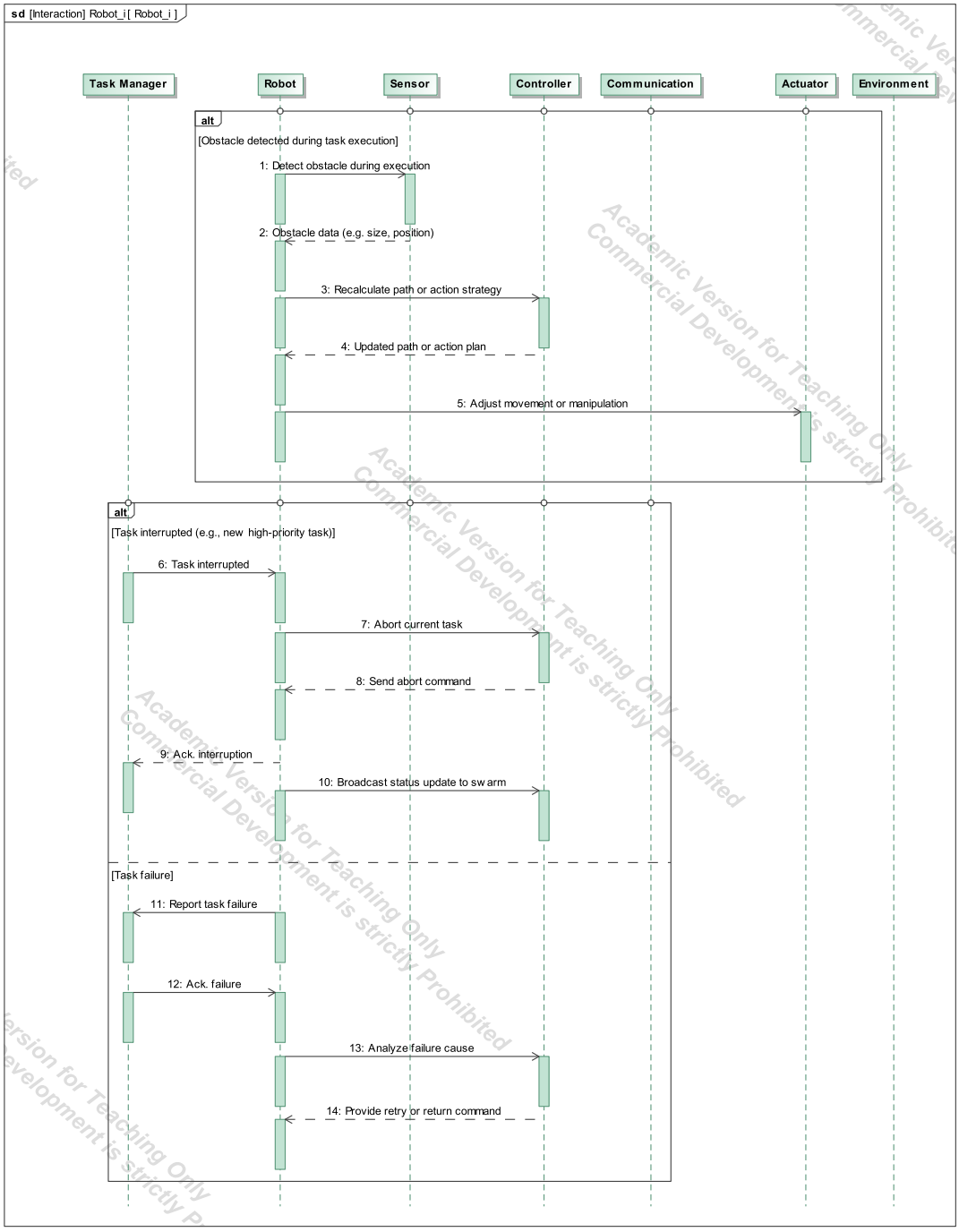

FIGURE 7. Robot interaction sequence for execution, feedback, and status reporting

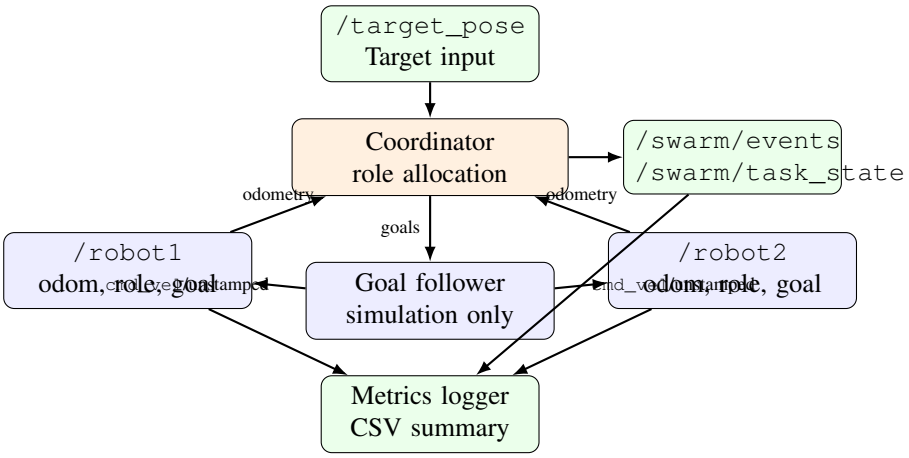

FIGURE 8. Software architecture for the implemented two-TurtleBot4 mission

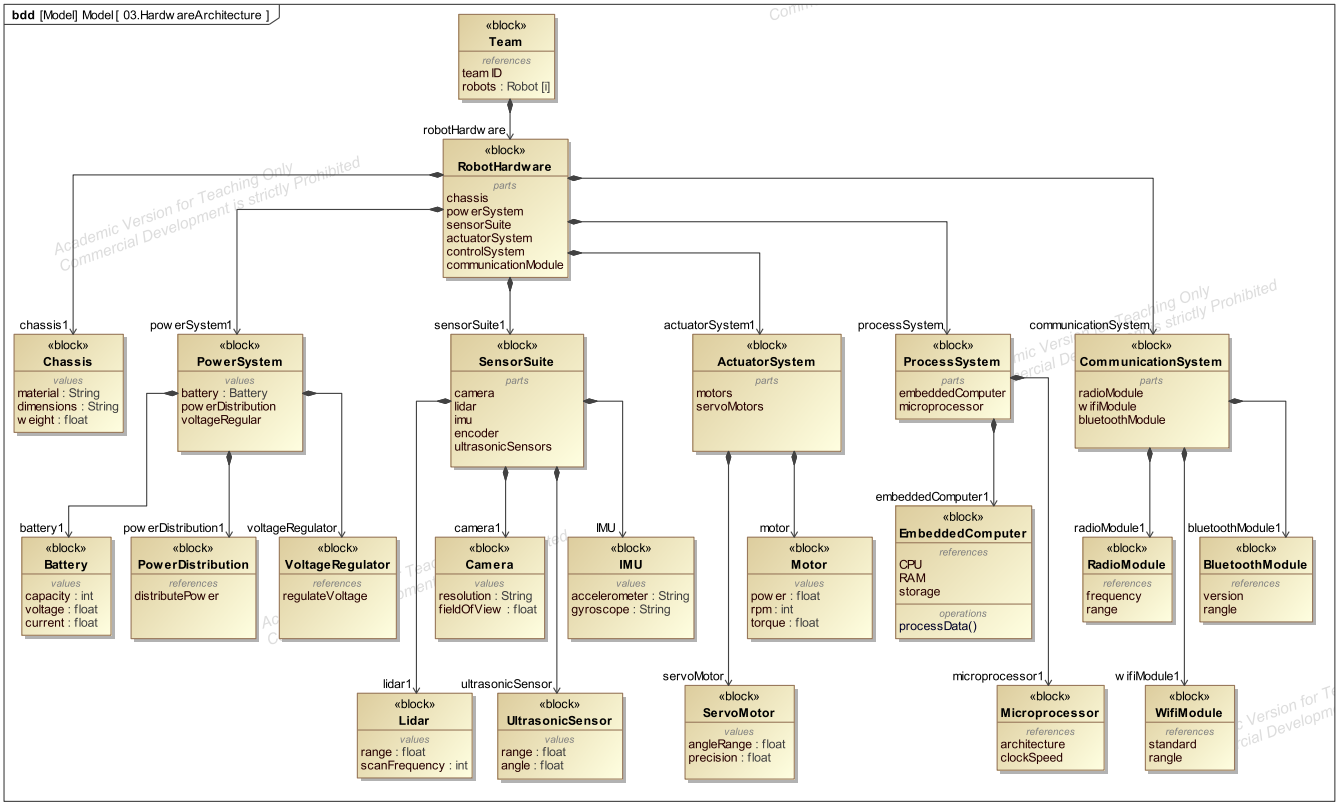

FIGURE 9. Hardware architecture model for one TurtleBot4-class robot

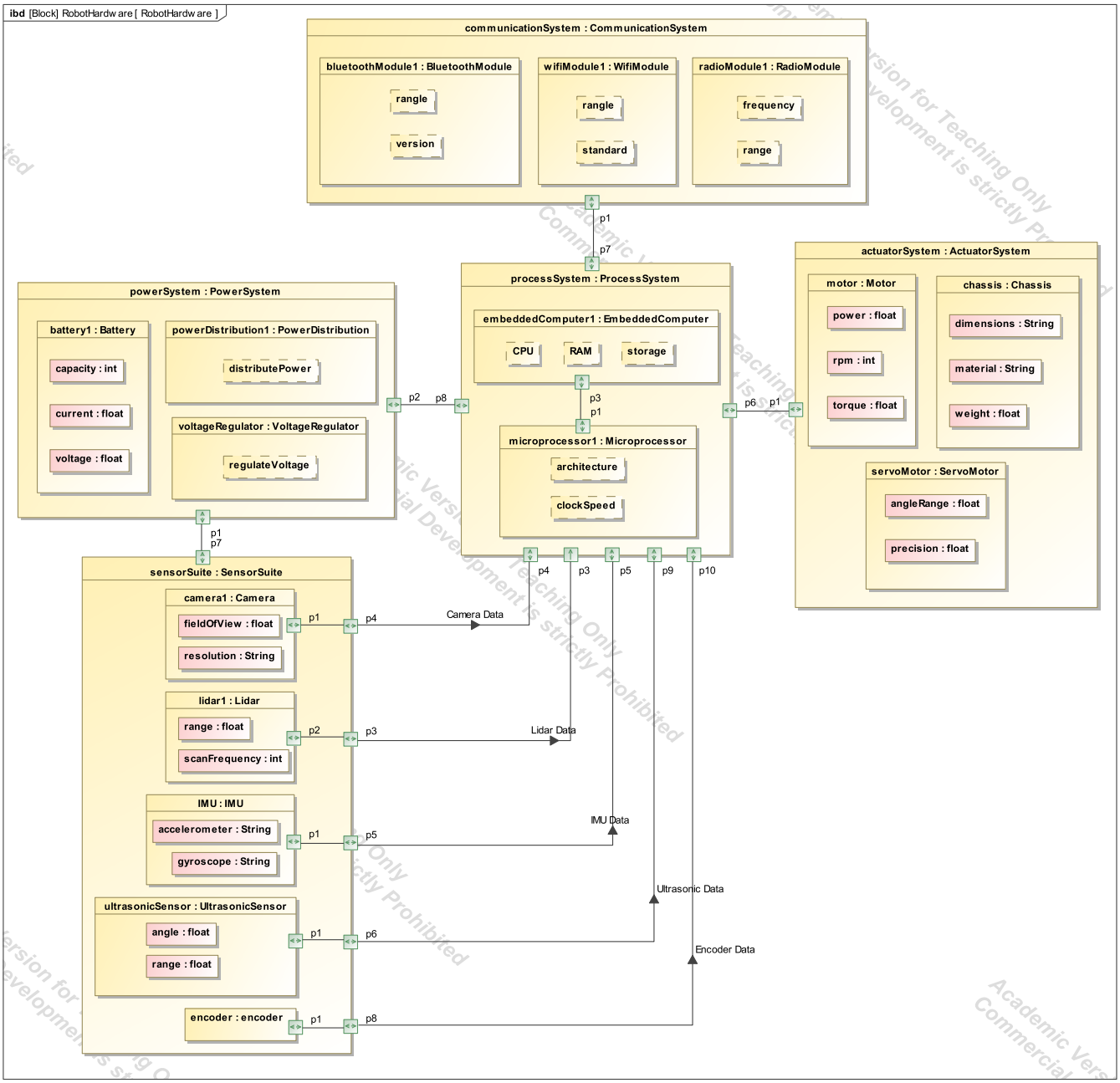

FIGURE 10. Hardware data-flow model using an Internal Block Diagram

E. SIMULATION AND PHYSICAL DEPLOYMENT MODES

Two simulation modes are provided. The stable mode uses lightweight differential-drive physics with TurtleBot4 visual meshes. This mode is the preferred validation path on the current machine because full Gazebo rendering with the official TurtleBot4 stack previously crashed in the renderer. The official mode launches the installed TurtleBot4 simulator twice and is kept server-only by default.

The physical mode does not launch Gazebo and does not command TurtleBot4 motion by default. It starts the coordinator and metrics nodes against the canonical `/robot1` and `/robot2` interfaces. The lab baseline for each TurtleBot4 Lite is FastDDS with `ROS_DOMAIN_ID=0`, Simple Discovery, and no user namespace; in that single-robot form, the robot exposes root topics such as `/odom`, `/scan`, `/oakd/rgb/preview/image_raw`, and `/cmd_vel_unstamped`. Two physical robots cannot both remain unnamespaced in the same ROS graph without topic collisions, so the deployment must either configure unique namespaces or provide an explicit bridge/remap layer

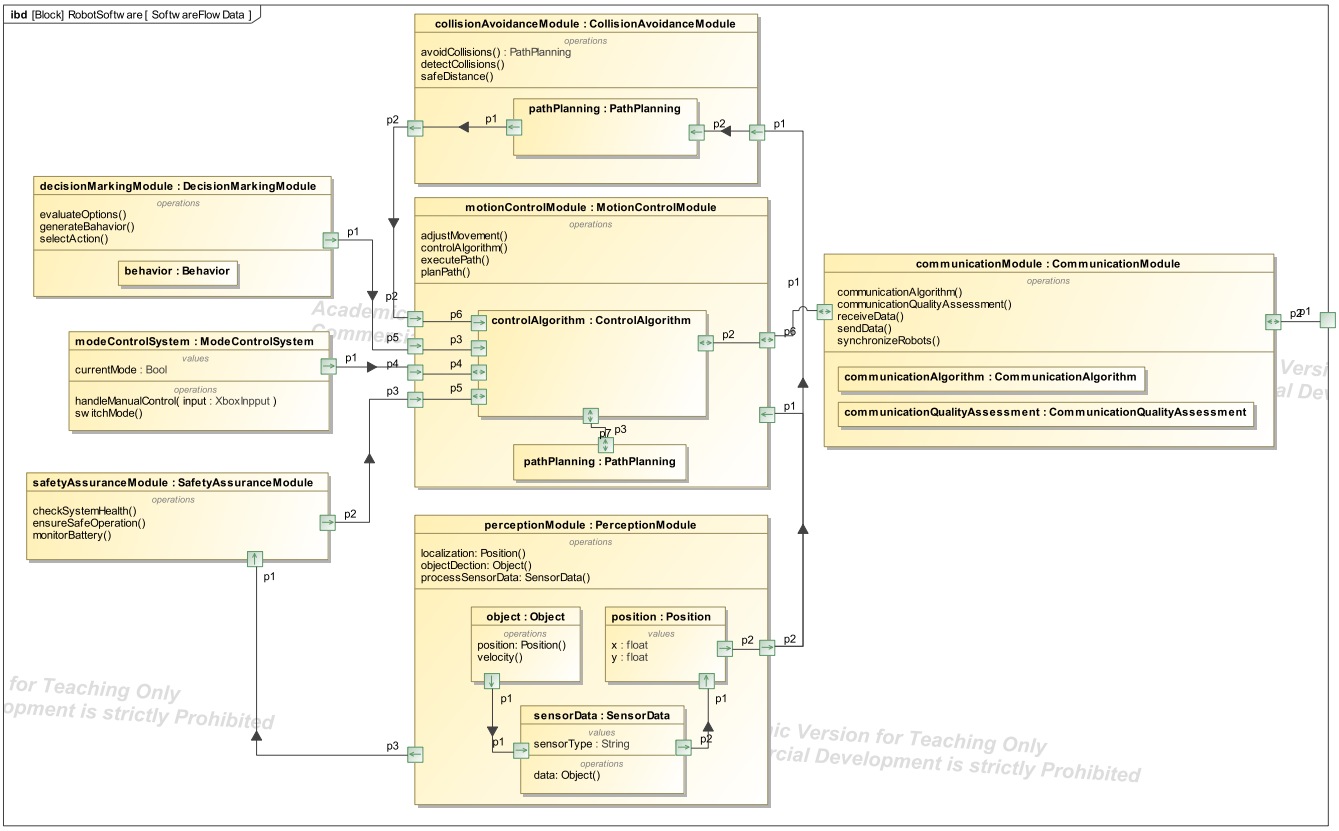

FIGURE 11. Software data-flow view for one robot

before running the two-robot coordinator. Direct velocity following can be enabled only by setting an explicit launch argument during controlled lab testing, and the physical command topic is `/robot*/cmd_vel_unstamped`. This separation is intentional: the paper validates the model-to-code chain in simulation while preserving a conservative path for physical experiments.

F. SIMULATION RESULT

A stable simulation run was executed with the two TurtleBot4 namespaces and the attacker/supporter mission. The latest checked run produced the summary shown in Table 2. The result validates the executable chain for target publication, odometry ingestion, role assignment, goal publication, robot motion, task completion, and metric logging.

TABLE 2. Stable two-TurtleBot4 simulation result

Metric	Value
Success	1
Duration	8.028 s
Role switches	0
Minimum separation	0.6985 m
`/robot1` path	1.3808 m
`/robot2` path	0.7805 m
Requirement refs	REQ-001–REQ-006

G. DISCUSSION

The result supports the paper's central claim: the same mission boundary defined in SysML is executable in ROS 2 and produces measurable validation evidence. The zero role-switch count indicates that the hysteresis margin prevented unnecessary reassignment during this run. The minimum separation remained positive, and both robot path lengths were recorded, confirming that the metrics logger observed the team-level behavior required by REQ-004.

The result should be interpreted within its intended scope. The simulation validates the traceability chain and the two-robot coordination behavior; it does not yet prove robustness across many environments, nor does it replace physical safety validation. For this reason, the physical launch mode keeps direct motion disabled unless the operator explicitly enables it.

V. CONCLUSION AND DISCUSSION

This paper presented a SysML-to-ROS 2 workflow for a two-TurtleBot4 target-acquisition mission. The workflow connects requirement, behavioral, structural, software, hardware, and data-flow models to executable ROS 2 nodes, topics, launch files, DDS settings, and metrics. The implemented case study assigns attacker and supporter roles from fresh odometry, publishes observable mission state, runs in simu-

lation, and preserves the canonical `/robot1` and `/robot2` interfaces that can be connected to physical TurtleBot4 Lite robots through namespace or bridge adaptation.

The main limitation is that validation remains limited to a two-robot target-acquisition task and to the stable simulation mode. Future work will connect the physical TurtleBot4 robots through the real launch mode, replace the simple simulation follower with a safety-aware navigation stack, and evaluate the same traceability process across repeated trials and more varied environments.

REFERENCES

[1] P. De Saqui-Sannes, R. A. Vingerhoeds, C. Garion, and X. Thirioux, "A taxonomy of MBSE approaches by languages, tools and methods," *IEEE Access*, 2022.

[2] Z. Li, G. Wang, J. Lu, D. G. Broo, D. Kiritsis, and Y. Yan, "Bibliometric analysis of model-based systems engineering: Past, current, and future," *IEEE Transactions on Engineering Management*, 2024.

[3] L. Bayındır, "A review of swarm robotics tasks," *Neurocomputing*, vol. 172, pp. 292–321, Jan. 2016.

[4] M. Brambilla, E. Ferrante, M. Birattari, and M. Dorigo, "Swarm robotics: a review from the swarm engineering perspective," in *Swarm Intelligence*, 2013.

[5] K. Giles and K. Giammarco, "A mission-based architecture for swarm unmanned systems," *Systems Engineering*, vol. 22, pp. 271–281, Feb. 2019.

[6] L. Bassi, C. Secchi, M. Bonfe, and C. Fantuzzi, "A SysML-Based and methodology for manufacturing and machinery modeling and design," *IEEE/ASME Transactions on Mechatronics*, no. Yang., 2011.

[7] R. Cloutier, B. Sauser, M. Bone, and A. Taylor, "Transitioning systems thinking to model-based systems engineering: Systemigrams to SysML models," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 2015.

[8] L. Li, N. L. Soskin, A. Jbara, M. Karpel, and D. Dori, "Model-based systems engineering for aircraft design with dynamic landing constraints using object-process methodology," *IEEE Access*, 2019.

[9] Z. Cui, M. Luo, C. Zhang, Y. Hua, R. Xie, R. Yu, and C. Huang, "MBSE for civil aircraft scaled demonstrator requirement analysis and architecting," *IEEE Access*, 2022.

[10] S. Gao, W. Cao, L. Fan, and J. Liu, "MBSE for satellite communication-system architecting," *IEEE Access*, no. develop, 2019.

[11] X. Zhang, B. Wu, X. Zhang, J. Duan, C. Wan, and Y. Hu, "An effective mbse approach for constructing industrial robot digital twin system," *Robotics and Computer-Integrated Manufacturing*, vol. 80, p. 102455, Apr. 2023.

[12] T. Winiarski, "MeROS: SysML-Based metamodel for ROS-Based systems," *IEEE Access*, 2023.

[13] E. T. Reniers, J. Kok, J. van de Mortel-Fronczak, and M. van de Molengraft, "Synthesis-based engineering of supervisory controllers for ROS-based applications," *Control Engineering Practice*, 2023.

[14] duy12i1i7, "TurtleBot4 Lite Lab Setup Guide." https://github.com/duy12i1i7/tb4-lite, 2026. Accessed: 2026-05-25.

[15] TurtleBot4 Documentation, "TurtleBot4 Setup Tool." https://turtlebot.github.io/turtlebot4-user-manual/software/turtlebot4_setup.html, 2026. Accessed: 2026-05-25.